

Analysis of Algorithms Laboratory

Lab: Insertion Sort, Binary Search, Interpolation Search, and Reading

Aispur P. Santiago

School of Mathematical and Computational Sciences

Yachay Tech University

San Miguel de Urcuquí - Imbabura, Ecuador

March 16, 2026

1 Introduction

This lab covers three tasks: (1) Insertion Sort for alphabetic characters, (2) Binary Search and Interpolation Search over a list of 10,000,000 numbers with execution time measurement, and (3) a summary of Kahn (1962) on topological sorting.

2 Problem 1: Insertion Sort on Alphabetic Characters

2.1 Algorithm Idea

Insertion Sort builds a sorted prefix. At step i , the character $arr[i]$ is inserted into the correct position among $arr[0..i-1]$ by shifting larger elements to the right.

2.2 Adjustment for Characters

No changes are required. Comparisons such as $arr[j] > key$ work for characters due to their ordered code values. With uppercase letters A–Z, the output is alphabetical order.

2.3 Complexity

- Worst-case time: $O(n^2)$
- Best-case time: $O(n)$
- Space: $O(1)$ extra space

2.4 Example Run

For $n = 10$, a sample output was:

Unsorted array:

J S V E N D E O Y L

Sorted array:

D E E J L N O S V Y

Insertion Sort time: 0.0009 ms

3 Problem 2: Binary Search vs Interpolation Search

3.1 Algorithm Idea

A list of 10,000,000 integers is prepared in non-decreasing order, then a key is searched.

Binary Search: Repeatedly halves the search interval.

Interpolation Search: Estimates the next position based on the values at the interval ends.

3.2 Sorted Input Requirement

Both searches require sorted input. The program generates a non-decreasing sequence directly (no `std::sort`).

3.3 Complexity

- Binary Search time: $O(\log n)$
- Interpolation Search average time (uniform data): $O(\log \log n)$
- Interpolation Search worst-case: $O(n)$
- Space: $O(1)$ extra space

3.4 Empirical Results

The program was compiled and executed with $n = 10,000,000$ and $\text{key} = 45$. Table 1 reports the measured times.

Table 1: Search execution time for $n = 10,000,000$

Search	Time (microseconds)
Binary Search	1.9
Interpolation Search	0.6

3.5 Discussion

For this run, Interpolation Search was faster than Binary Search. This is consistent with the expected behavior when the data distribution is close to uniform, where Interpolation Search tends to reduce the search interval more aggressively.

4 Problem 3: Reading Summary (Kahn, 1962)

This section summarizes Arthur B. Kahn (1962), “*Topological sorting of large networks*”.

Kahn presents a method to obtain a topological order for very large directed networks, motivated by practical needs in PERT (Program Evaluation Review Technique). At the time, existing tools often had capacity limits around 5,000 to 10,000 elements, and manual numbering was common but difficult to maintain. Large networks are revised often, and small changes can force renumbering across large parts of the network.

A topological ordering is described as a list in which every path moves forward in the list: no node appears before its predecessors. Kahn emphasizes network terminology common in PERT: events (nodes), activities (directed links), and predecessor/successor relations.

The core idea is a ranking-based procedure that processes all routes in parallel, rather than relying on threading techniques. The algorithm keeps two main structures: an activity list grouped by predecessor, and an event list that stores for each event the current count of immediate predecessors not yet processed. The method starts by finding all events with predecessor count zero. These are assigned sequence numbers and marked as processed. For each processed event, the predecessor counts of its successors are decremented. When a successor count reaches zero, it becomes eligible to be processed in the next step.

Kahn also discusses memory and time. The design is built to fit internal memory efficiently, with compact information per event and activity. The paper provides a running-time estimate based on the size of the network (events and activities) and a parameter related to how far out of order labels appear along a route. Example estimates show that networks with about 30,000 activities can be processed within minutes for typical cases, with total processing time including preparation around 40–50 minutes on an IBM 7090.

The algorithm terminates when an iteration produces no new zero-count events. At that point, either all events are processed (success), a directed cycle exists (illegal loop), or the network must be segmented due to size. For debugging cycles, Kahn suggests reporting partially affected events, running a reverse pass, and detecting “adverse labels” where a predecessor has a higher label than its successor.

Finally, the paper notes an alternative application: ordering glossary definitions so that each term is defined using only previously defined terms, producing a textbook-style order.

Repository

Full source code available at: [Aispur Santiago - Analysis of Algorithms, GitHub Repository](#)